Eigen Decomposition

 # Diagonal Matrix:- A diagonal matrix has non-zero elements only on the main diagonal.

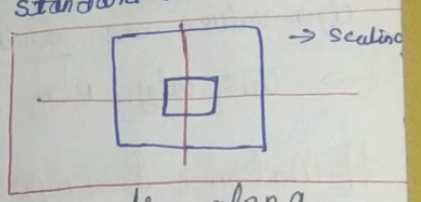
$$D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

→ Easy to multiply (in normal matrix, very difficult) $\Rightarrow D^{100} = \begin{bmatrix} 3^{100} & 0 & 0 \\ 0 & 5^{100} & 0 \\ 0 & 0 & 2^{100} \end{bmatrix}$

→ Easy to invert

→ Eigen values = diagonal elements

→ Eigen vectors = standard basis vectors



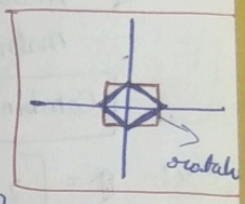
 # Why important in ML

- PCA diagnolizes covariance matrix
- Diagonal matrix = independent features
- Scaling features uses diagonal matrices

→ A diagonal matrix represents independent scaling along each axis.

 # Orthogonal Matrix :- (Perpendicular (90°))

A matrix Q is orthogonal if $Q^T Q = Q Q^T \Rightarrow I$



→ Perfect rotation, no scaling or shearing

→ $Q^{-1} = Q^T$ {Transpose and Inverse are same}

→ Preserves distance, preserves dot product, $\det = \pm 1$

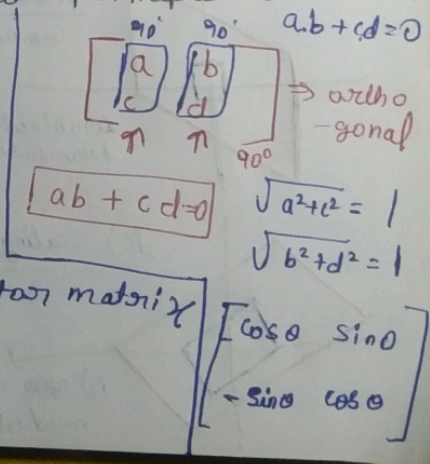
 # Why important:-

→ Eigen vectors of covariance matrix form an orthogonal matrix

→ PCA = rotate data into new orthogonal axes.

→ PCA uses orthogonal eigenvectors so that principal components are uncorrelated

Property	Diagonal	orthogonal
mainuse →	Scaling	Rotation
Inverse →	Easy	Transpose
PCA role →	Variance matrix	Eigenvector matrix
Geometry →	Stretch (increase, decrease)	Rotate



Eigen Decomposition

 # Diagonal Matrix:- A diagonal matrix has non-zero elements only on the main diagonal.

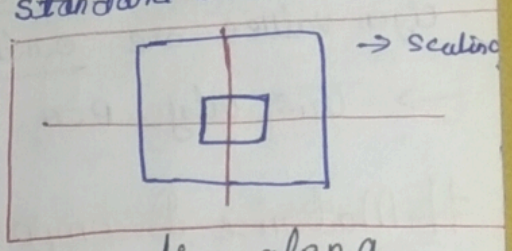
$$D = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 2 \end{bmatrix}$$

→ Easy to multiply (in normal matrix, very difficult) $\Rightarrow D^{100} = \begin{bmatrix} 3^{100} & 0 & 0 \\ 0 & 5^{100} & 0 \\ 0 & 0 & 2^{100} \end{bmatrix}$

→ Easy to invert

→ Eigen values = diagonal elements

→ Eigen vectors = standard basis vectors



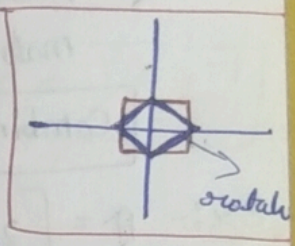
 # Why important in ML

- PCA diagnolizes covariance matrix
- Diagonal matix = independent features
- Scaling features uses diagonal matrices

→ A diagonal matrix represents independent scaling along each axis.

 # Orthogonal Matrix :- (Perpendicular (90°))

A matrix Q is orthogonal if $Q^T Q = Q Q^T \Rightarrow I$



→ Perfect rotation, no scaling or shearing

→ $Q^{-1} = Q^T$ {Transpose and Inverse are same}

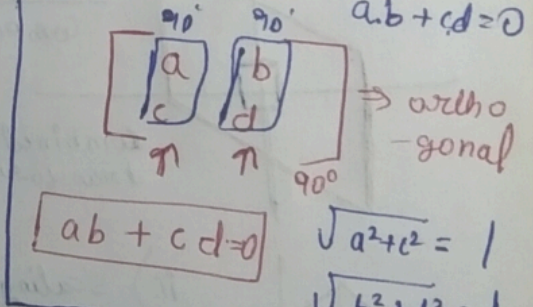
→ Preserves distance, preserves dot product, $\det = \pm 1$

 # Why important:-

→ Eigen vectors of covariance matrix form an orthogonal matrix

→ PCA = rotate data into new orthogonal axes.

→ PCA uses orthogonal eigenvectors so that principal components are uncorrelated



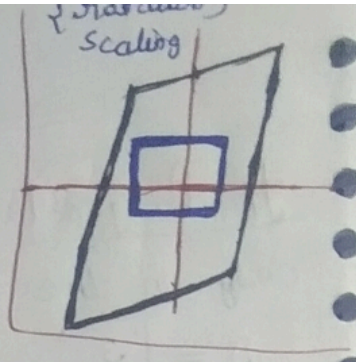
Property	Diagonal	orthogonal
main use →	Scaling	Rotation
Inverse →	Easy	transpose
PCA role →	Variance matrix	Eigenvector matrix
Geometry →	Stretch (increase, decrease)	Rotate

$$\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

Symmetric Matrix :-

→ A matrix A is Symmetric if : $\boxed{A = A^T}$

Ex: $A = \begin{bmatrix} a & c \\ c & b \end{bmatrix} = A^T \Rightarrow \begin{bmatrix} a & c \\ c & b \end{bmatrix}$



→ **Real Eigen Values :-** All Eigen values of a symmetric matric are real number (not complex nums)

→ **Orthogonal Eigenvectors :-** Eigen vectors corresponding to different eigen values are orthogonal.

→ This why PCA gives Perpendicular principle components.

Matrix Decomposition :-

→ Appling one transformation after another

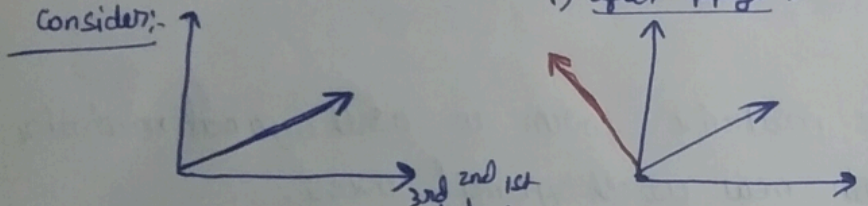
→ if matrix A = first transformation
matrix B = second transformation

$\boxed{\text{Combined Transformation} = BA}$ (from right to left)

Ex:- $A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$ $B = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$

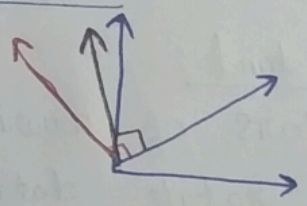
Apply A → then B $BA = \begin{bmatrix} 0 & -2 \\ 2 & 0 \end{bmatrix}$ rotation & Scaling

Consider:-



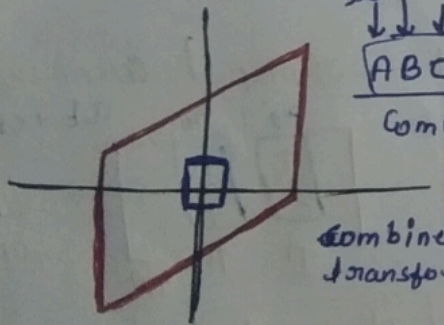
i) after applying B

ii) then A



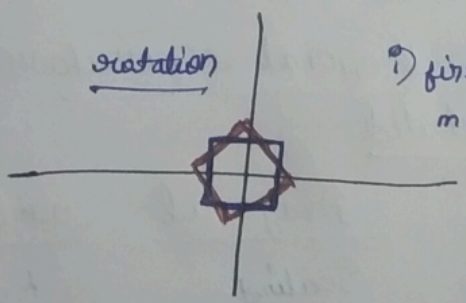
$\boxed{ABC = D}$

Composition



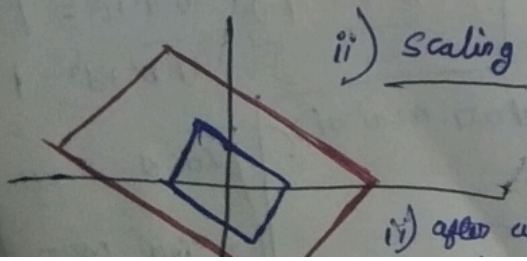
combined transformation

rotation

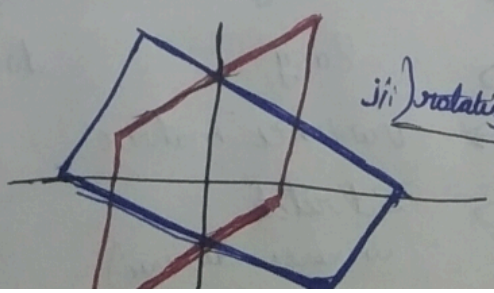


i) first applying C matrix, (rotation)

ii) scaling



iii) rotating



ii) after applying A matrix

iv) after applying

Matrix Decomposition:-

- LU Decomposition
- QR decomposition
- (Eigen decomposition
- SVD) mainly used in ML

Matrix decomposition means breaking complex matrix into simpler matrices.

Why do we do decompose matrices:- To simplify computations, understand data structure and solve ML Problem Efficiently.

Eigen decomposition $\Rightarrow$ $A = V \Lambda V^{-1}$ $\Rightarrow$ $\begin{bmatrix} a & b \\ c & d \end{bmatrix} = \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix} \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix} \begin{bmatrix} x_1 & x_2 \\ y_1 & y_2 \end{bmatrix}^{-1}$

A (2×2 matrix) = V (2 Eigen vectors) Λ (Diagonal Eigen values)

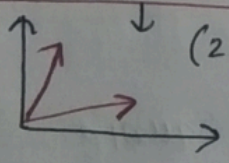
- V is a matrix whose cols are Eigen Vectors of A
- Λ (Lamba) → diagonal matrix of Eigen values.
- V^{-1} is the inverse matrix

- Eigen Vectors ⇒ directions
- Eigen values ⇒ Strength along that direction

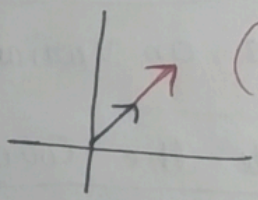
from $A\vec{V} = \Lambda\vec{V}$ to $A = V\Lambda V^{-1}$

assuming:-

1. Square Matrix :- Eigen decomposition is only defined for square matrix
2. Diagonalizability :- for (n*n) matrix, it should have $\frac{n}{2}$ linearly independent eigen vectors. (2×2)



(2 linear independent eigen vectors)



(linear but not independent Eigen vectors

Eigen decomposition of Symmetric matrix:

[Spectral Decomposition] $\Rightarrow$ $A = V \Lambda V^{-1}$

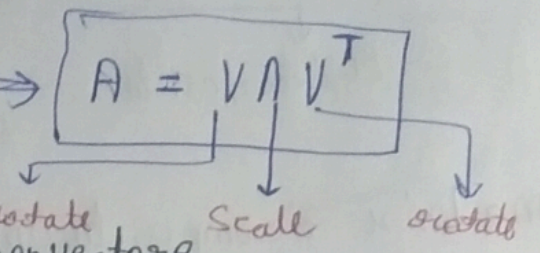
- V → linear transformation → rotates (orthogonal)
- Λ → (diagonal) scaling
- V^{-1} → inverse rotation (orthog

analyze this diagrams for clearer understanding

→ If A is a real Symmetric matrix

$$A = A^T$$

Then its eigen decomposition is ⇒ $A = V \Lambda V^T$



- A ⇒ Symmetric matrix
- V ⇒ matrix of orthonormal eigenvectors
  - Columns of V are Eigenvectors ($V^T = V^{-1}$)
  - Eigen vectors are perpendicular (orthogonal) to each other.
- Λ ⇒ diagonal matrix of Eigen values.
  - Each ~~value~~ Eigen diagonal value corresponds to an Eigen Vector in V
    (two thing go together / linked together)

→ one Eigen value tells how much scaling happens along its Eigen vectors

How Eigen Values & Eigen Vectors are used in PCA :-

Goal of PCA :- Reduce dimensions, keep max information (variance), Remove redundancy (correlated features)

Step 1 :- mean-center the data

given data, matrix X (cols = features, rows = samples)

$$X_{centered} = X - mean(X)$$

- Pca works on variance, so Centering is required

Step 2 : Compute the Covariance matrix :-

$$Cov = \frac{1}{n-1} X^T X$$

→ covariance matrix is square
→ Covariance matrix is Symmetric

Step 3 : Eigen decomposition of Covariance :-

since Covariance matrix is Symmetric :

$$Cov = V \Lambda V^T$$

V → Eigenvectors ⇒ Principal directions
Λ → Eigenvalues ⇒ Variance along those directions

Step 4: Eigen Vectors in PCA

- Each Eigen vectors gives a new axis
- axes are orthogonal.
- These axes are called Principal Components (PCs)
- Eigen Vectors ⇒ directions of maximum variance

Eigen values in PCA

- Eigen value ⇒ amount of variance along that Eigenvector
- Larger eigenvalue ⇒ more information
- PCA sorts Eigen values in descending order

Step 5: Top K Components:-

- Choose top k components with largest Eigen values

Step 7:- Project data on new axes.

$$X_{PCA} = X_{centered} V_k$$

This is dimensionality reduction

Intuition:-

→ PCA rotates axes
→ find directions where data data spreads the most
→ drop directions with very little spread (noise)

{Eigen vectors becomes principal components (PC's) and Eigen values represent variance along them. We keep top Components to reduce dimensionality while preserving information}

Advantages of Eigen decomposition:-

- ML → PCA
- Physics
- quantum physics
- signal theory

Ex:- A $(n \times n)$ Square matrix

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$\Rightarrow A^{1000} = ?$

$A^{1000} \overset{\text{decompose}}{=} V\Lambda V^{-1} V\Lambda V^{-1} \ldots$

$= V \boxed{\Lambda^{100}} V^{-1}$

Easy, when diagonal

$=$ 2 matrix multiplication

PCA Variants

Standard PCA

- is linear
- works on dense, numeric, low noise data
- uses Covariance matrix

But in real data can be:

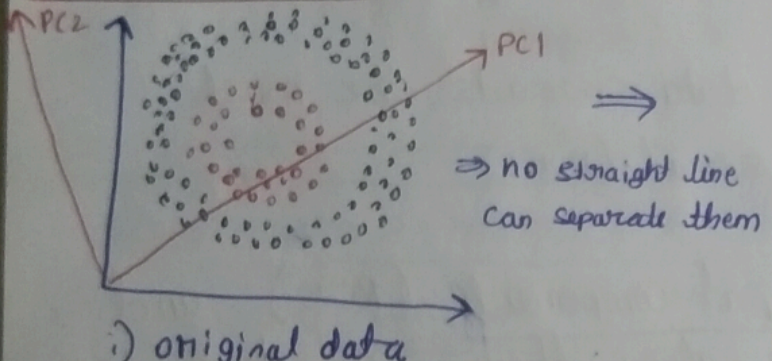
- non-linear
- very large
- noisy, sparse, streaming

So variants Exists

PCA variants

Standard PCA (Eigen PCA)	SVD-based	Kernel PCA	Incremental	Sparse	Randomised
what:- Eigen decomposition	• Large data	• non-linear data	• PCA in batches	• forces sparsity in components	• Approximate PCA using randomness
use:- • small medium data	# why like	limitation	use:- large dataset	• slower than PCA	→ Extremely large dataset
limitation:- Need full Covariance matrix	• faster • No Cov matrix needed	• slow	Streaming data		→ slight loss in accuracy

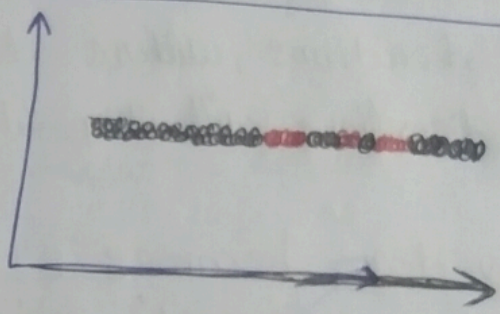
Kernel PCA :-



⇒ no straight line can separate them

i) original data

Projecting data after PCA is 1D



PCA fails

→ PCA assumes that principal components are a linear combination of the original features. It can't handle complex polynomial relationships between features

idea:- if data is not separable in low dimensions, map it to a higher dimensions where it becomes separable.

Common Kernel functions

i) Linear kernel ii) Polynomial kernel iii) RBF (Gaussian) Kernel

iv) sigmoid kernel

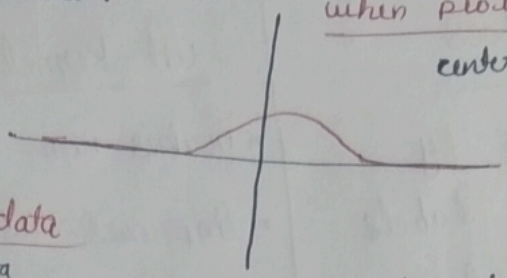
mathematic function ⇒ $y = e^{-x^2}$ ⇒ rbf

→ A Kernel is a function that helps linear algorithms to works with non-linear data.

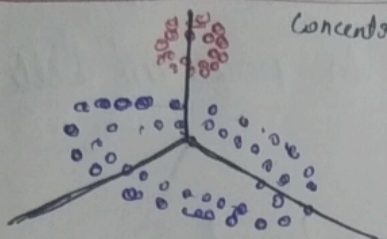
→ It does this by computing similarity between data points as if they were mapped to a higher-dimensional space, without actually creating new features.

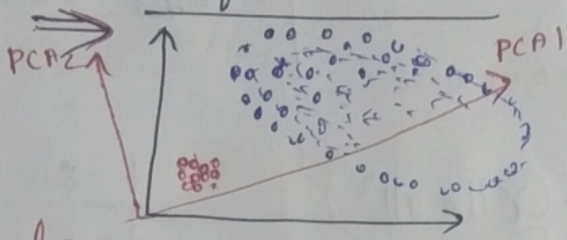
→ Kernel PCA use this idea to capture non-linear patterns that normal PCA cannot.

⇒ $y = e^{-x^2}$ ⇒

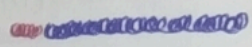


for the circular / spiral data





in 1D



How Kernel PCA works :-

Start with original data :-

→ If patterns are non-linear (circles, curves), Normal PCA fails

Step 2 :- Choose a kernel function :- Common choice (RBF - Gaussian) Kernel

. The kernel measures similarity b/w 2 points

. Acts as if data is mapped to a higher dimension

→ No actual feature Expansion happens

Ex :-

	f1	f2	f3	f4	fn
f1	-	-			
f2					
f3					
f4					
fn					

Step 3 :- Compute the kernel (similarity) matrix

$$K_{ij} = K(x_i, x_j)$$

→ Each value tells how similar two data points are

→ matrix size ⇒ [no. of samples × no. of samples] (n×n) : no interpretability : no scalability

step 4 :- Center the kernel matrix

Just like mean-centering data in PCA :

→ required to apply PCA logic correctly

Step 5:- Eigen decomposition of

$$K = V \Lambda V^T$$

→ Eigenvector ⇒ non-linear principal components
→ Eigen values ⇒ importance (variance captured)

Step:- Project data
Select top k Eigenvectors
Project data to get lower dimensional representation

Wt Kernel PCA does

→ Does not classify
→ Does not use labels
→ It is unsupervised

Wt Kernel PCA helps with

- makes non-linear data more separable
- Improves visualization
- Helps classifier to perform better

how Similarity measured :-

Ex:- **RBF Kernel**

$$K(x,y) \Rightarrow e^{-(\gamma \|x-y\|^2)}$$

gamma

uses ⇒ euclidean distance
small distance → large similarity
large distance → similarity ≈ 0

Ex:- **linear kernel**

$K(x,y) \Rightarrow x \cdot y$

→ not distance base
→ measures similarity via angle / alignment